

Lecture Notes for Neural Networks I

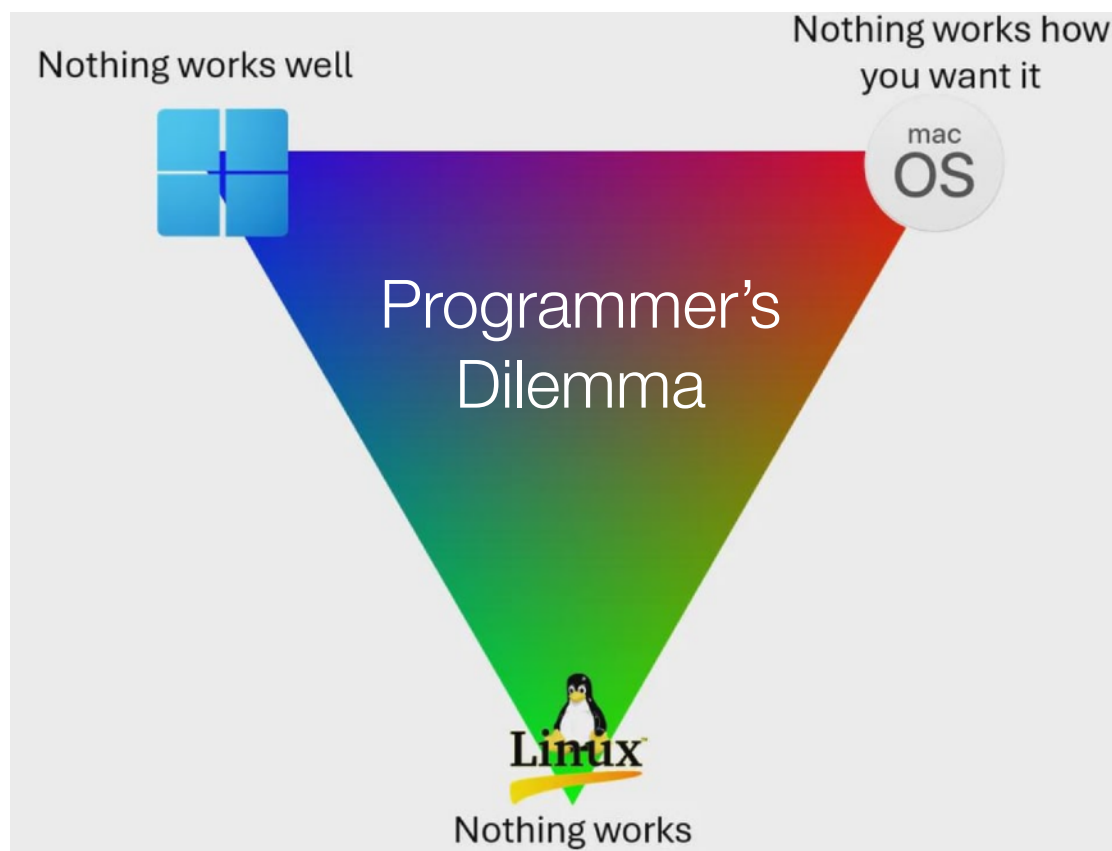


Preprocessing and Visualization

Class Logistics and Agenda

- Logistics:
 - Participation / Teams
 - Be sure you look at **Lab One!**
 - Office Hours
- Agenda
 - Data Exploration
 - Data Preprocessing
 - Data Visualization

Dataset Selection Lab One



Class Overview, by topic

Table Data
Visualization

Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Linear and
Logistic
Regression

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Neural Networks
and Back Prop.

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Wide and Deep
Networks

Convolutional
Networks

Sequential
Networks

Keras, Tensorflow, Torch
Intuition, Detailed implement.

Large Language
Models (if time)

Hugging Face
Intuition only

Last Time

- Datatypes
- Imputation
- *Some Document Features*

Loading the Titanic Data for Example Visualizations

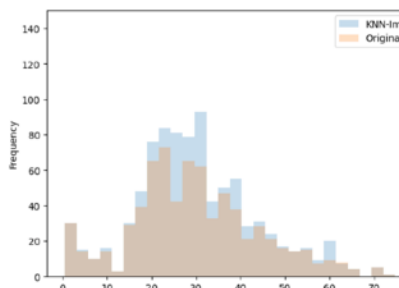
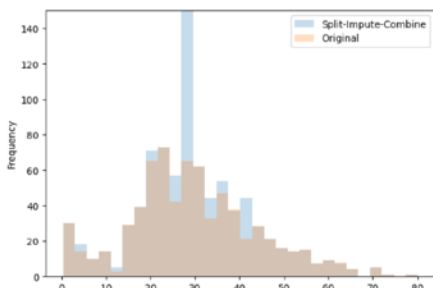
```
l: # load the Titanic dataset
import pandas as pd
import numpy as np

print('Pandas:', pd.__version__)
print('Numpy:', np.__version__)

df = pd.read_csv('https://raw.githubusercontent.com/ericlarson/dataMiningN
df.head()
```

```
l: # note that the describe function defaults to using only some variable
df.describe()
```

```
l: print(df.dtypes)
print('=====')
print(df.info())
```



K-Nearest Neighbors Imputation

TID	Pregnant	BMI	Age	Diabetes
1	Y	33.6	31-40	positive
2	N	26.6	31-40	negative
3	Y	23.3	?	positive
4	?	28.1	21-30	negative
5	N	43.1	31-40	positive
6	Y	25.6	21-30	negative
7	Y	31.0	21-30	positive
8	Y	35.3	?	negative
9	N	30.5	51-60	positive
10	Y	37.6	21-30	positive

For $k = 3$, find 3 closest neighbors

TID	Preg.	BMI	Age	Diabetes	Distance d_k
3	Y	23.3	?	positive	0
6	Y	25.6	21-30	negative	$(0 + 2.3 + 1)/3$
2	N	26.6	31-40	negative	$(1 + 3.3 + 1)/3$
4	?	28.1	21-30	negative	$(4.8 + 1)/2$

... repeat for all rows, select 3 closest ...

Imputed Age: 21-30

Distance can be calculated differently:

- Difference for valid features only
- May need to normalize ranges
- Weight neighbors differently?
- Have min # of valid features?
- Type: Euclidean, city-block, etc.

$$d_k = \frac{1}{|F_{\text{valid}}|} \sum_{i \in F_{\text{valid}}} \|f_i - f_i^{(k)}\|$$

i th feature, f , in row

PascalCase for Classes, Structs, etc.

Python, Java, C#, Swift

snake_case for Method/Property:

Python, Rust

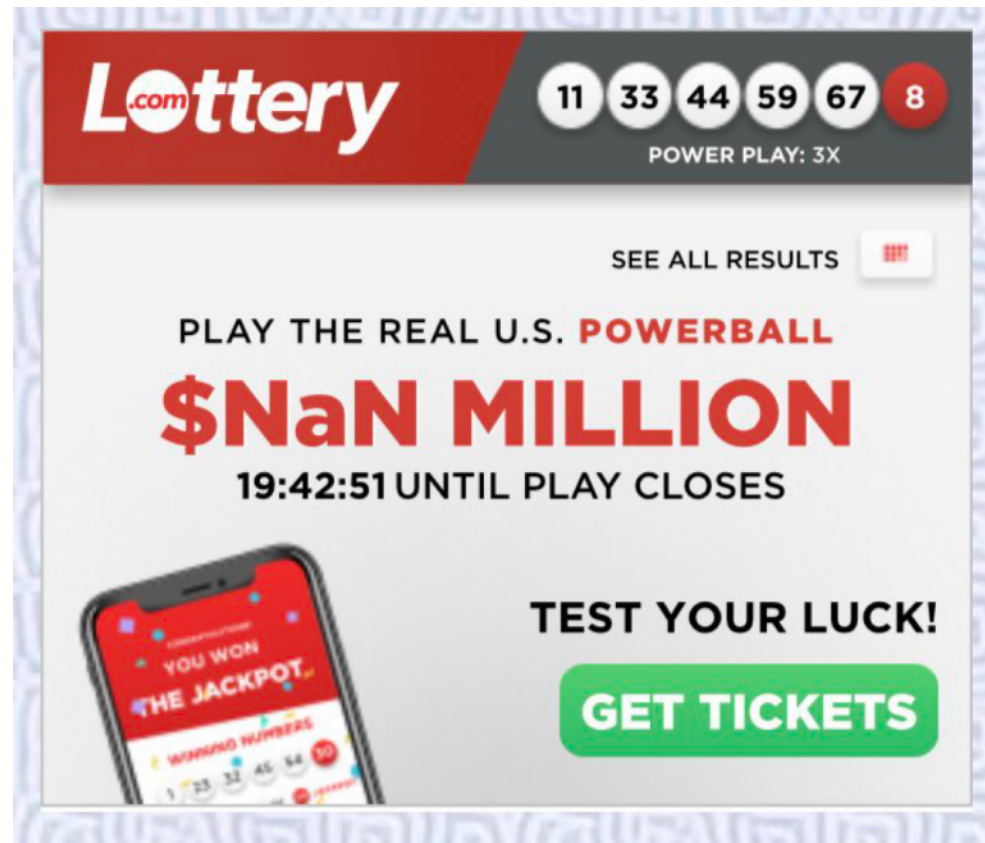
camelCase for Methods/Property:

Java, C#, Swift, ...

C++: all over the place...

Data Exploration

Practical Data Quality:
Remember to Ignore
NaNs in Aggregation

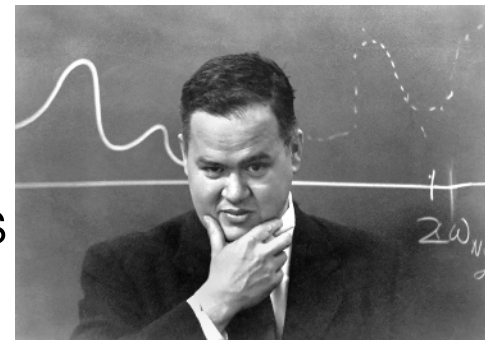
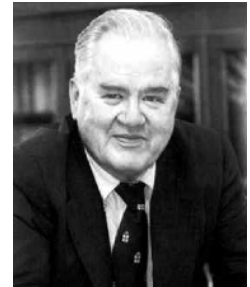


What is data exploration?

Must know the **Business/Policy Understanding** before Exploring!

Data Exploration: Generic methods for understanding the data distributions and trends

- Helps to guide preprocessing and analysis
- Exploratory Data Analysis (EDA) by Dr. John Tukey:
 - Tukey's take: Visualizing, Clustering and Anomaly detection
- Larson's take:
 - Feature statistics, aggregations
 - Visualizations without complicates questions
 - Examples:
 - Will we impute? Any obvious outliers?
 - How is target distributed?

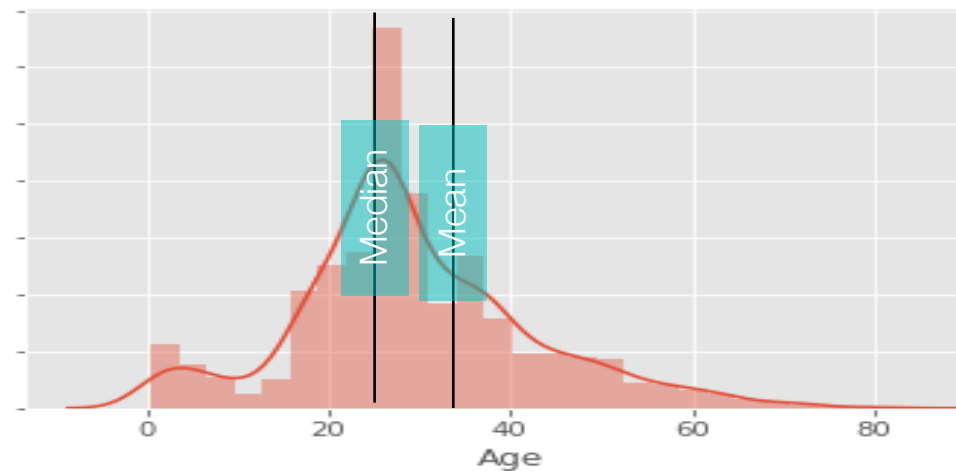


Summary Statistics of Features

- frequency, location, and spread
 - Examples: location by **mean or percentile** (numeric)
spread by **standard deviation** (numeric)
frequency by **mode** (categorical)
- Most summary statistics can be calculated in a single pass through the data

$$\text{sample mean}(x) = \mu_x = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\text{sample median}(x) = x_{50\%}$$



Measures of Spread

- **Dynamic Range** (max - min), e.g., 0-65 years
- The **variance** or standard deviation is the most common measure of the spread of a set of points.

$$\text{sample var}(x) = \sigma_x^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \mu_x)^2$$

- σ^2 can be sensitive to outliers, so other measures are also popular:

Average Absolute Difference

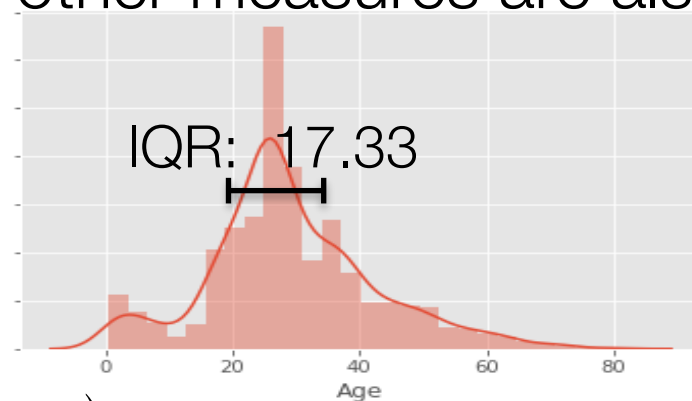
$$\text{AAD}(x) = \frac{1}{N} \sum_{i=1}^N |x_i - \mu_x|$$

Median Absolute Difference

$$\text{MAD}(x) = \text{median}(|x_1 - \mu_x|, \dots, |x_i - \mu_x|, \dots, |x_N - \mu_x|)$$

Interquartile Range

$$\text{IQR}(x) = |x_{75\%} - x_{25\%}|$$



STD: 13.89

AAD: 10.67

MAD: 8.29

Correlation, Pearson's Linear

$$\text{sample } \rho = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \cdot \sqrt{\sum_i (y_i - \bar{y})^2}}$$

$$\rho = \frac{\text{COV}(\mathbf{x}, \mathbf{y})}{\sigma_x \cdot \sigma_y}$$

=0, *uncorrelated*: x has no influence on y

=1, *correlated*: +/-change in x also means +/-change in y

= -1, *anti-correlated*

$$\cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

uncentered correlation is cosine similarity

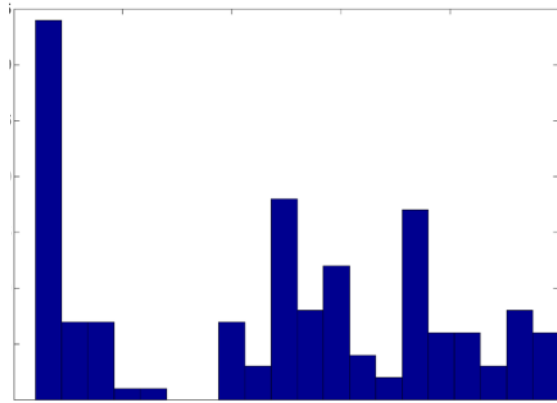
$$\cos(\theta) = \rho$$

if $\mathbf{x}, \mathbf{y}$ zero mean



Self Test 2a.1

What measure of **spread** is **most appropriate** for the data in the histogram below?



- A) Standard Deviation
- B) Interquartile Range
- C) Median Absolute Difference
- D) None of these

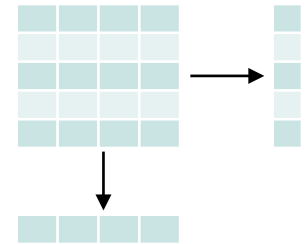
Data Preprocessing



Common Preprocessing Techniques

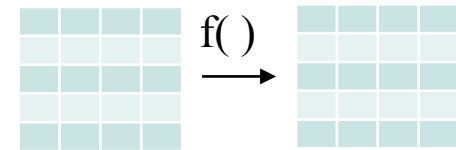
- **Aggregation: Combine features/samples**

- ◆ Reduce the number of attributes or objects
- ◆ Aggregated data tends to be more stable



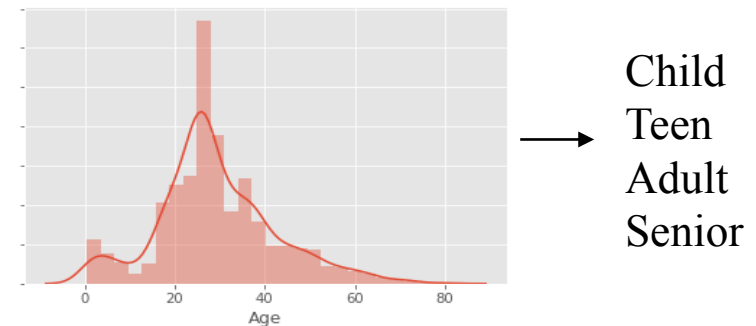
- **Transformation: Change of scale, range**

- ◆ Normalize dynamic ranges
- ◆ More numerically stable when combining



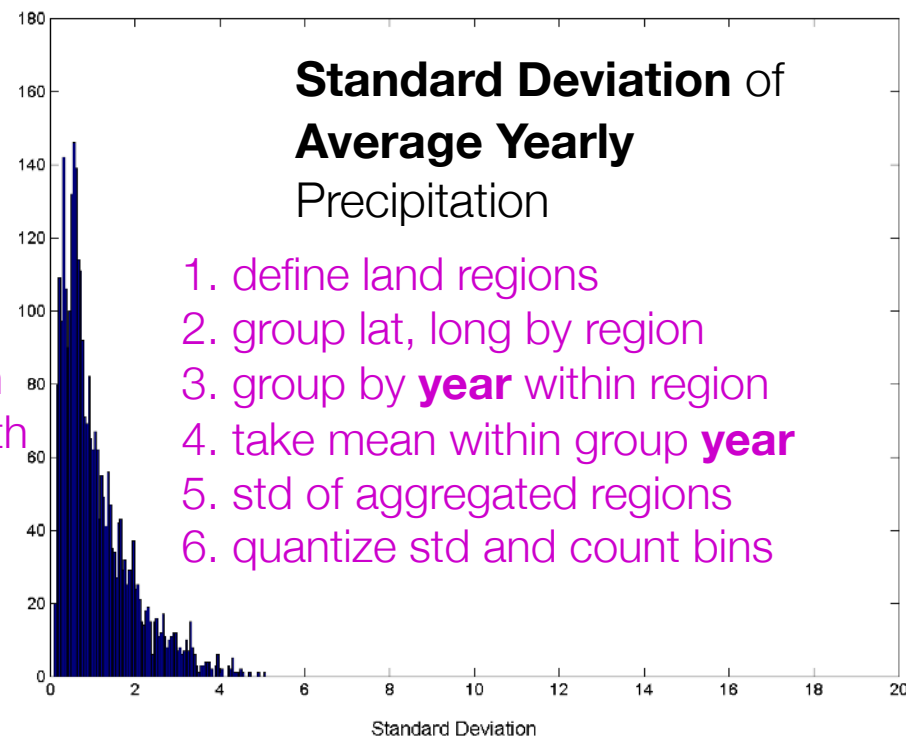
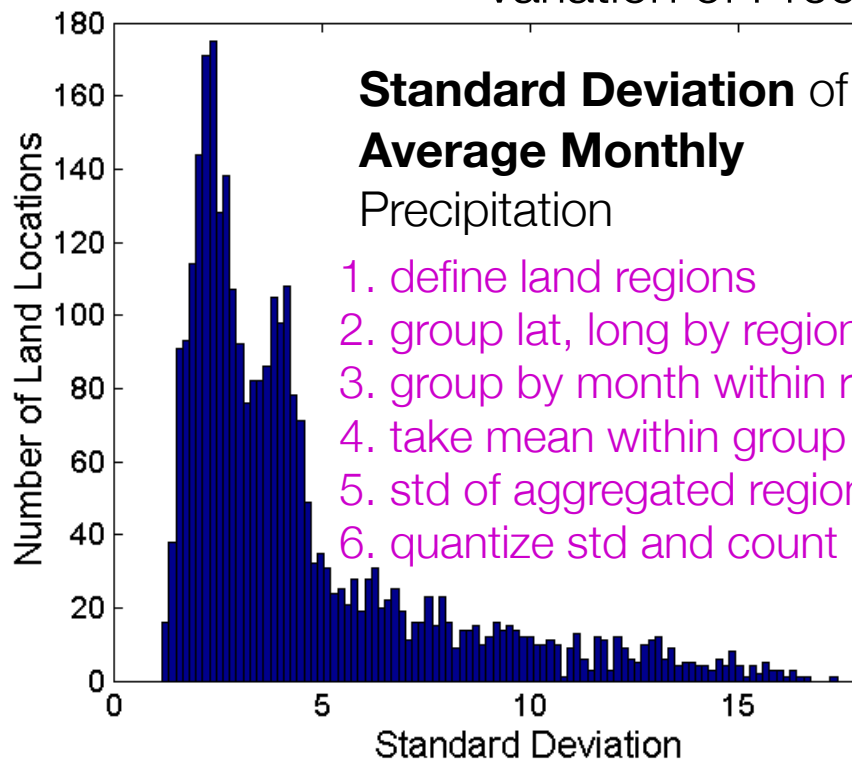
- **Quantization: Make discrete**

- ◆ More stable
- ◆ More semantically meaningful



Preprocessing: Aggregation

Variation of Precipitation in Australia



How has aggregation has been used to create these plots?

<i>TID</i>	<i>Location</i>	<i>time</i>	<i>measured rainfall</i>
1	<i>lat, long</i>	<i>measured daily</i>	<i>X.XX cm</i>

Preprocessing: Transformation

- Monotonically map one set of values to a set of replacement values

- **Standardization and Normalization**

- Z-scores `df_normalized = (df-df.mean())/(df.std())`

- min/max `df_normalized = (df-df.min())/(df.max()-df.min())`

Normalization options in scikit-learn:

<code>preprocessing.maxabs_scale(X, *[, axis, copy])</code>	Scale each feature to the [-1, 1] range without breaking the sparsity.
<code>preprocessing.minmax_scale(X[, ...])</code>	Transform features by scaling each feature to a given range.
<code>preprocessing.normalize(X[, norm, axis, ...])</code>	Scale input vectors individually to unit norm (vector length).
<code>preprocessing.quantile_transform(X, *[, ...])</code>	Transform features using quantiles information.
<code>preprocessing.robust_scale(X, *[, axis, ...])</code>	Standardize a dataset along any axis
<code>preprocessing.scale(X, *[, axis, with_mean, ...])</code>	Standardize a dataset along any axis.
<code>preprocessing.power_transform(X[, method, ...])</code>	Power transforms are a family of parametric, monotonic transformations that are applied to make data more Gaussian-like.

Attribute Transformation in Python

```
>>> from sklearn.preprocessing import scale, StandardScaler
>>> X = [[ 1., -1.,  2.],
...      [ 2.,  0.,  0.],
...      [ 0.,  1., -1.]]
>>> X_scaled = scale(X)
>>> X_scaled
array([[ 0. ..., -1.22...,  1.33...],
       [ 1.22...,  0. ..., -0.26...],
       [-1.22...,  1.22..., -1.06...]])
```

using direct
methods, **NO**

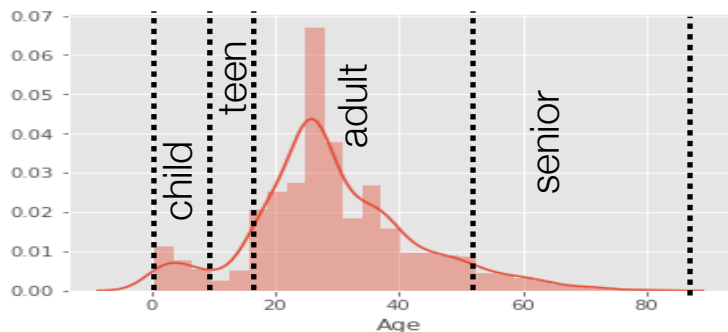
```
>>> scaler = StandardScaler().fit(X)
StandardScaler(copy=True, with_mean=True, with_std=True)
>>> scaler.mean_
array([ 1. ...,  0. ...,  0.33...])
>>> scaler.std_
array([ 0.81...,  0.81...,  1.24...])
>>> scaler.transform(X)
array([[ 0. ..., -1.22...,  1.33...],
       [ 1.22...,  0. ..., -0.26...],
       [-1.22...,  1.22..., -1.06...]])
```

using object
oriented approach
Preferred!!

```
>>> scaler = StandardScaler().fit(X_train)
>>> scaler.transform(X_test)
```

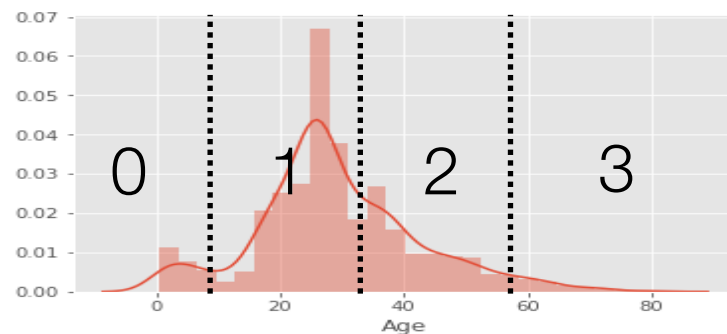

Preprocessing: Quantization

Expert selected



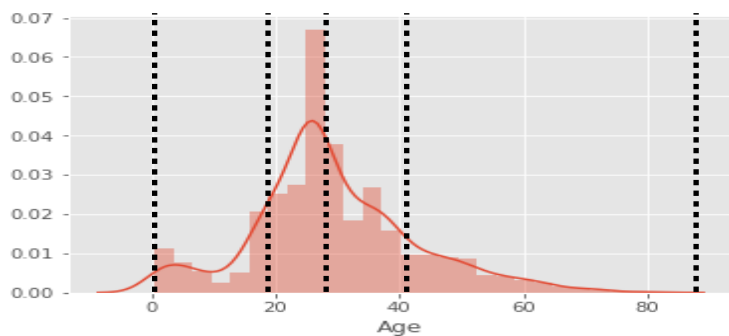
Data

strategy='uniform'



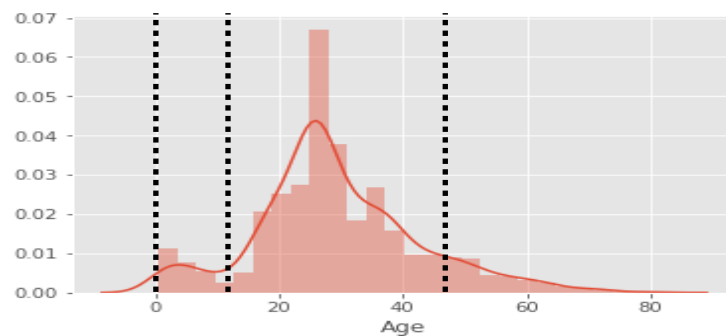
Equal interval width

strategy='quantile'



Equal frequency

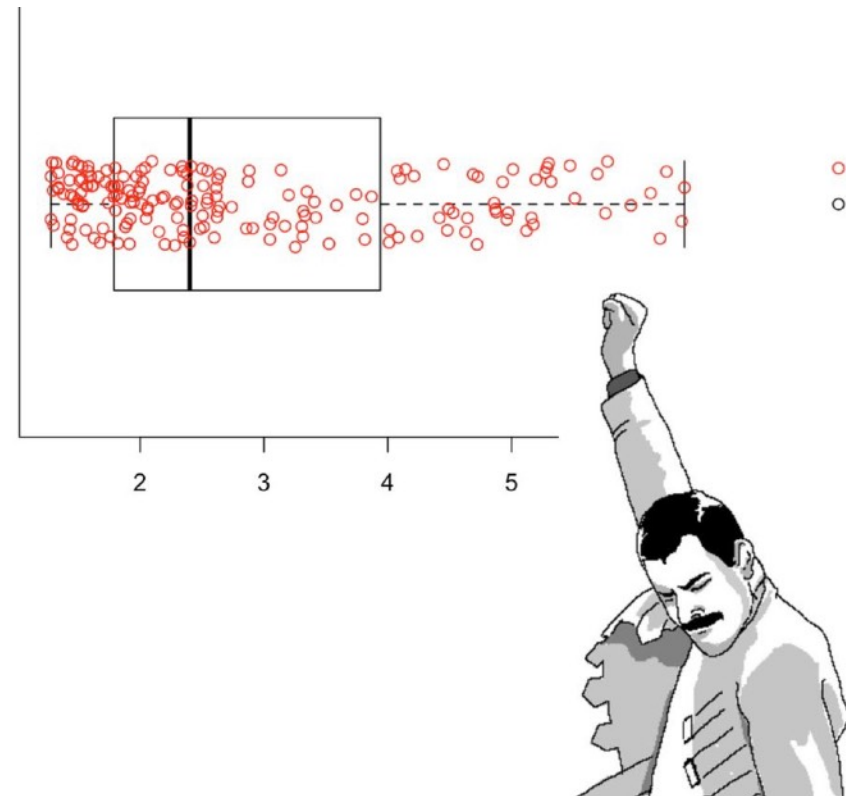
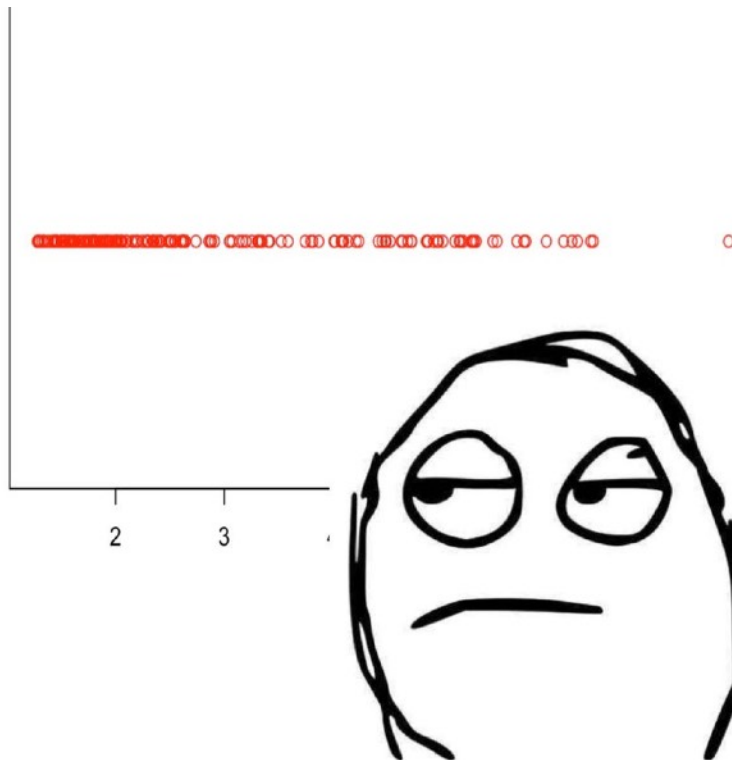
strategy='kmeans'



clustering: e.g., K-means

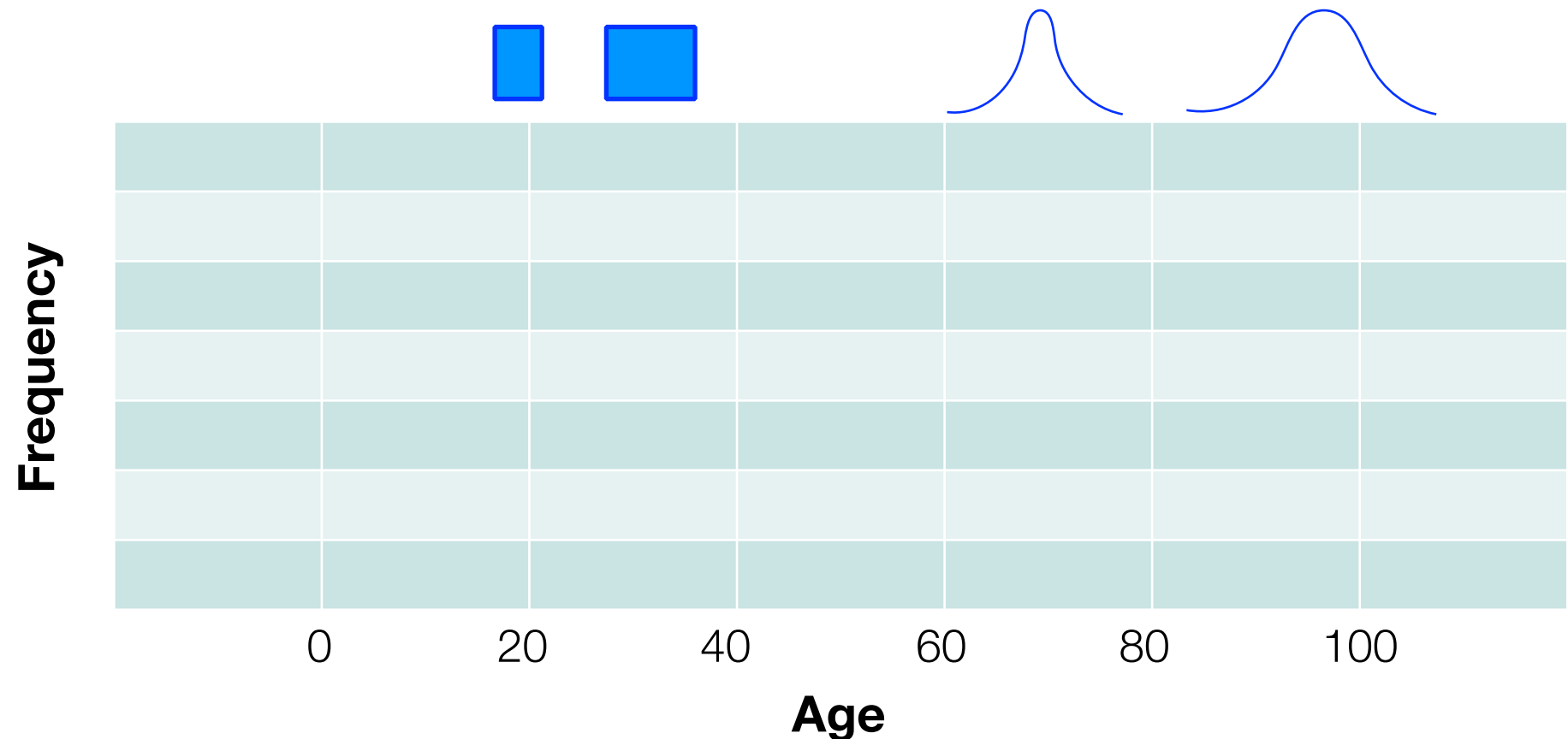
```
est = KBinsDiscretizer(n_bins=4,  
                        encode='ordinal', strategy='uniform')  
X_discrete = est.fit_transform(X)
```

Data Visualization

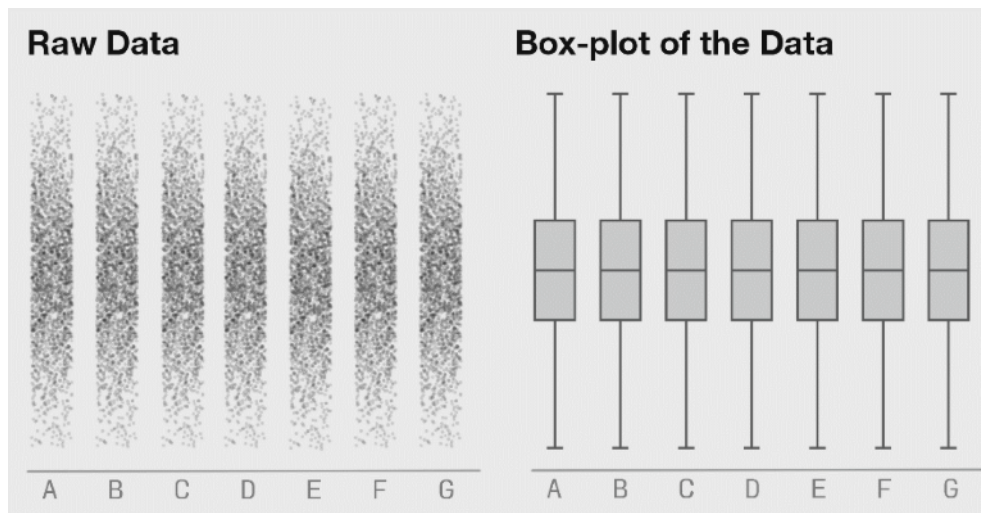


Histograms and Kernel Density

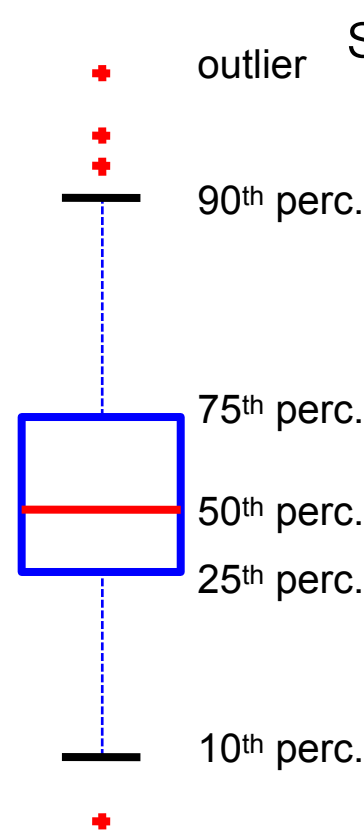
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked	2	3	male	Q	S	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S	0	1	1	0	1
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C	0	0	0	0	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S	0	1	0	0	1



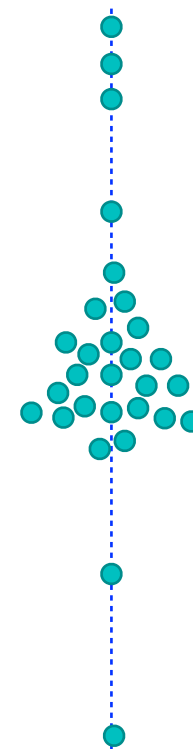
Visualization Techniques: Box, Violin, Swarm



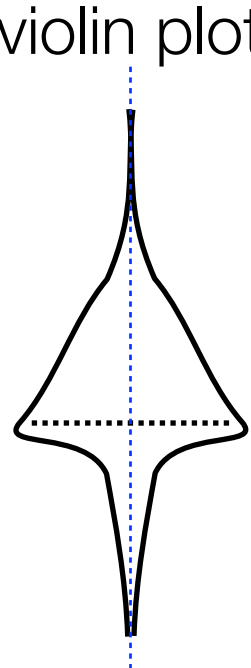
box plot



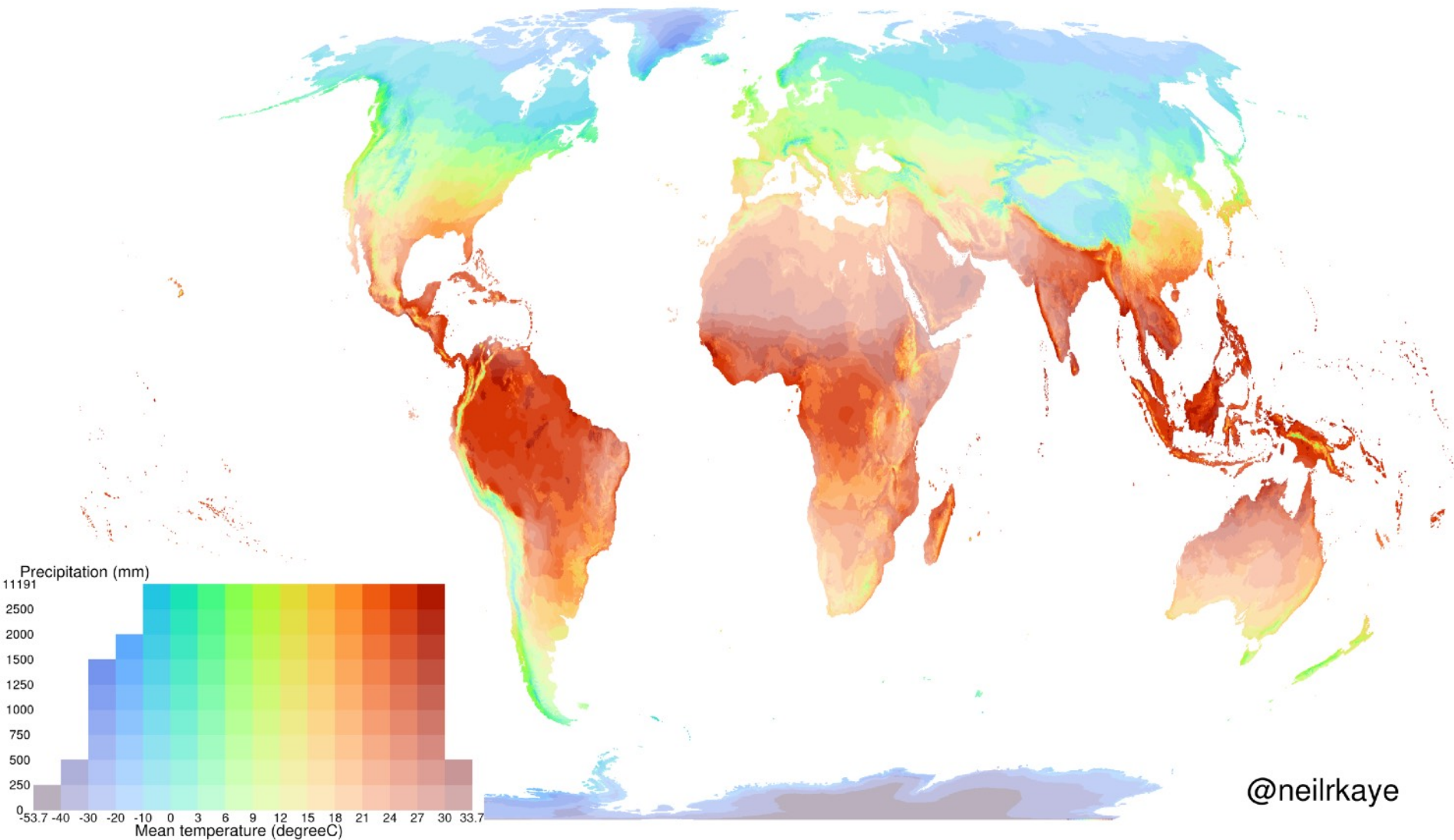
swarm plot



violin plot



Annual mean temperature and precipitation totals (long term average)



Choosing How/What to Visualize?

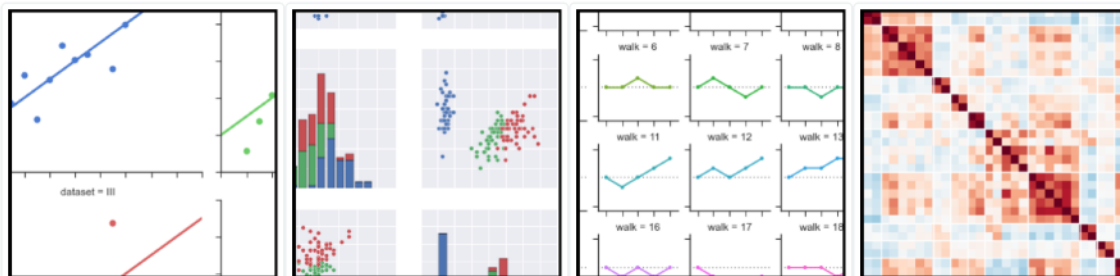
- Perform EDA, get the basics out of the way
- Look at business/policy for data and ask an **interesting** question
- Think about the **best plot** to answer the question
 - Do you have the **right data** for visualizing?
 - Do you need to **worry** about the **amount** of data in the plot (aliasing, low samples, etc.)?
 - Can your question be answered **reliably**?
- **Interpret** the visualization: Did it answer the question?
 - **No**: Think of another visual
 - **Kinda**: Ask a follow up question
 - **Yes**: Truly? Think more critically

Matplotlib

- Python plotting utility
 - Has **low level plotting** functionality
 - Highly **similar to Matlab and R** for plotting
- Extended to be visually more beautiful by
 - **seaborn**: stanford data visualization group
- Plotly: interactive visuals, via JS



Seaborn: statistical data visualization



- You tell me what conclusions we are getting from these graphs
 - Histogram and KDE
 - HeatMaps and Correlation
 - Scatter and Scatter Matrix
 - Box / Violin / Swarm



`02.Data Visualization.ipynb`

Matplotlib
Seaborn
Plotly

For Next Lecture

- Next Time:
 - Finish Visualization Demo
 - First Town Hall Meeting